LayerZero Stargate Credit Messaging Mintable Burnable

Security Assessment

April 23rd, 2026 — Prepared by OtterSec

Nicholas R. Putra nicholas@osec.io

Reyn Shaw reyn@osec.io

Table of Contents

01 — Executive Summary

Overview

Stargate protocol engaged OtterSec to assess the `credit-messaging-mintable-burnable-system` program. This assessment was conducted between April 12th and April 21st, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 1 finding throughout this audit engagement.

In particular, we advised rejecting minting or burning credits to `OFT` paths and when Eid is zero, because both are non-meaningful credit domains, rendering mint/burn operations no-ops that still emit misleading events (OS-LSC-SUG-00).

Scope

The source code was delivered to us in a Git repository at https://github.com/stargate-protocol/stargate-v2. This audit was performed against commit c2c9c7f.

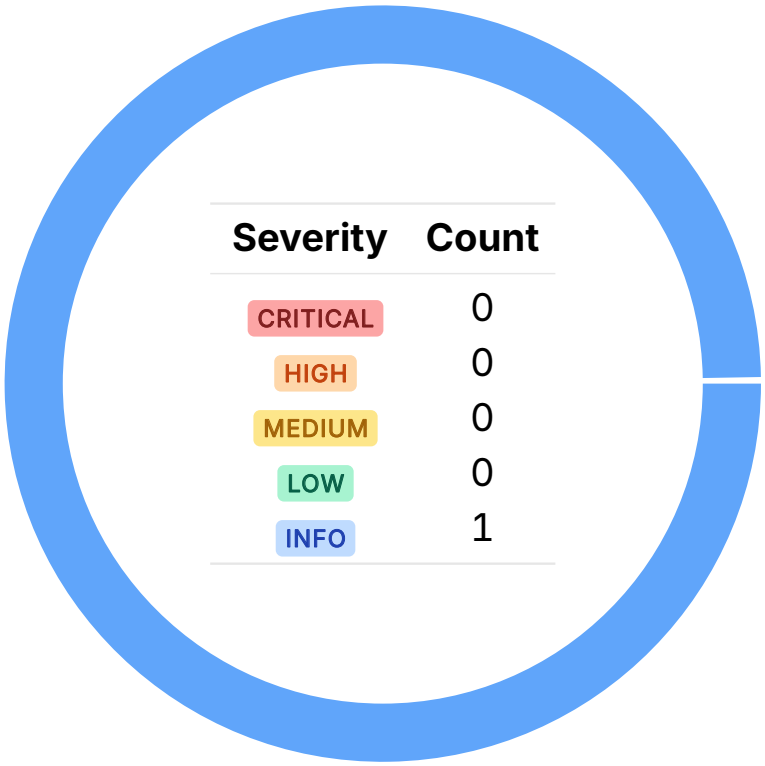
A brief description of the program is as follows:

Name	Description
credit-messaging-mintable-burnable-system	An owner-only emergency mechanism that locally mints or burns Stargate V2 path credits to restore or correct cross-chain capacity when credits become stranded or inconsistent. It addresses liveness failures such as stuck-but-safe liquidity, without directly moving funds or changing custody.

02 — Findings

Overall, we reported 1 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	0
INFO	1

03 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LSC-SUG-00	Recovery should reject `OFT` paths and `eid == 0` because both are non-meaningful credit domains, making mint/burn operations no-ops that still emit misleading events.

Failure to Reject OFT and Zero Eid Paths OS-LSC-SUG-00

Description

Currently, there is no check preventing `mintCredits` or `burnCredits` from targeting OFT-backed paths, which already operate with `UNLIMITED_CREDIT`. As a result, attempting to adjust credits there has no economic effect, so the operation is effectively a no-op. However, the recovery flow will still emit credit-related events, which is misleading as it falsely suggests that a meaningful credit repair occurred.

Similarly, allowing `mintCredits` or `burnCredits` to target `paths[0]` is meaningless since a zero `Eid` does not represent any chain and is only utilized internally as a local source identifier for recovery flows.

Remediation

Reject minting or burning credits to the path, which is an `OFT` and a path with `Eid` equal to zero.

Patch

The LayerZero team acknowledged this issue.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.